

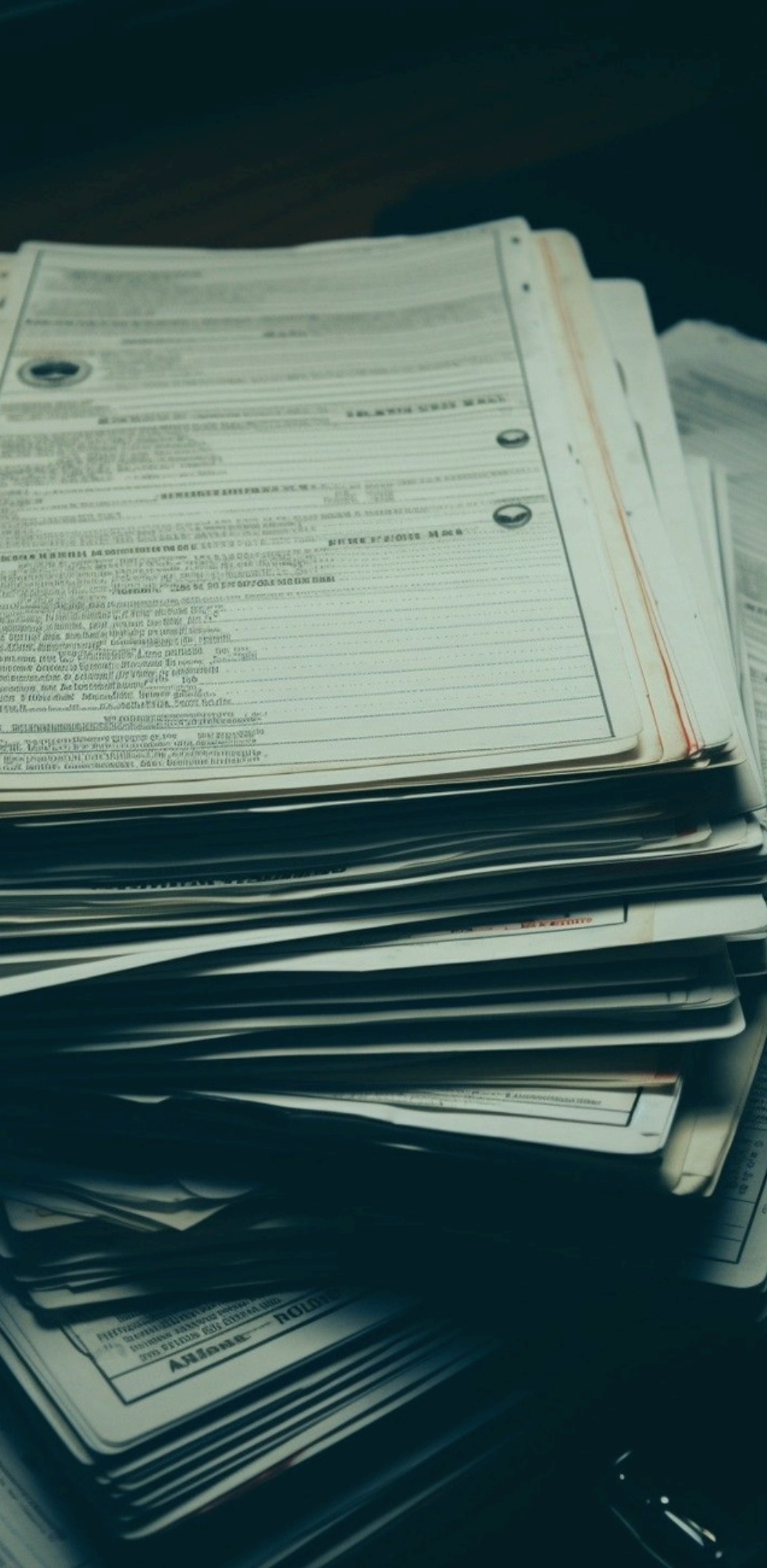
Sistema de Clínica

Sistema básico de clínica seguindo os **Princípios SOLID**

Raphaela Reis
Estudante

Marcos Pires
Estudante





Contexto do problema

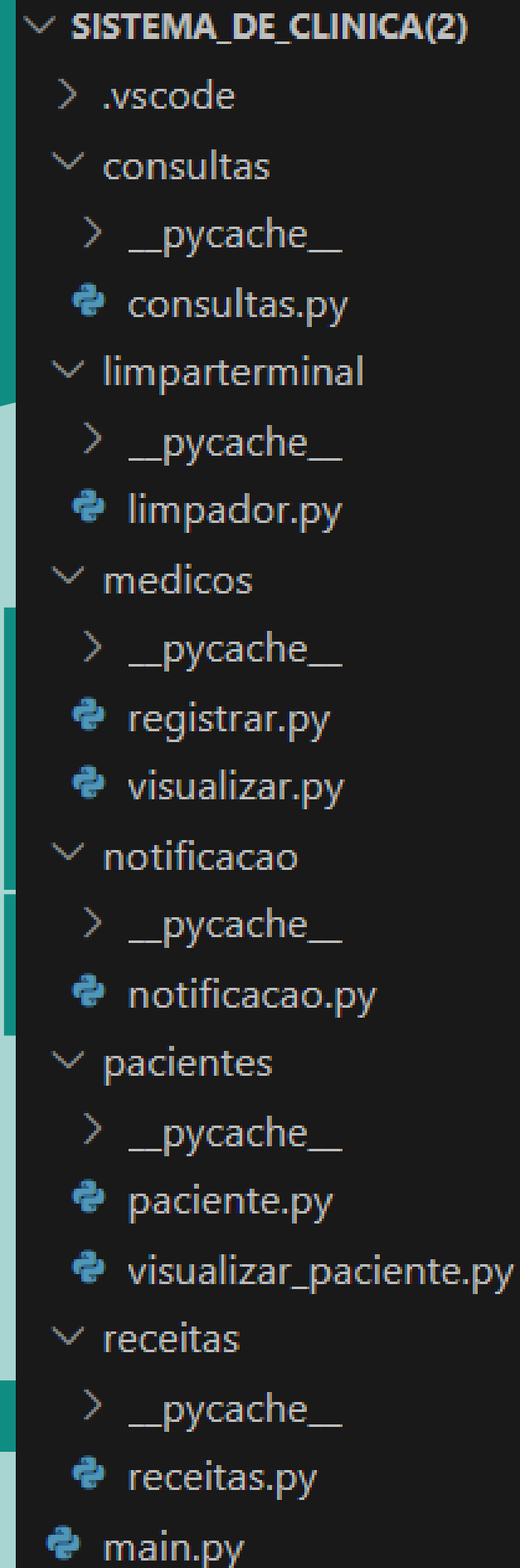
Clínicas médicas lidam diariamente com o cruzamento de muitos dados sensíveis, seja dos pacientes, seja dos médicos.

- Problema:** Falta de integração e sistemas suscetíveis a falhas
- humanas. Registros manuais permitem erros graves, como documentos inválidos ou datas impossíveis.
- Solução:** Um sistema de gestão integrado, focado na validação
- mais rigorosa de dados. Ele impede erros logo na entrada de dados e conecta os setores da clínica de forma segura.

Arquitetura e Separação de Responsabilidades

Arquitetura Modular

- O projeto foi dividido em pacotes/pastas (pacientes, medicos, consultas, receitas, notificacao) para evitar alto acoplamento.
- Uso de um arquivo main.py que apenas chama as funções, sem concentrar a lógica de negócio.



```
✓ SISTEMA_DE_CLINICA(2)  
  > .vscode  
  ✓ consultas  
    > __pycache__  
    📄 consultas.py  
  ✓ limparterminal  
    > __pycache__  
    📄 limpador.py  
  ✓ medicos  
    > __pycache__  
    📄 registrar.py  
    📄 visualizar.py  
  ✓ notificacao  
    > __pycache__  
    📄 notificacao.py  
  ✓ pacientes  
    > __pycache__  
    📄 paciente.py  
    📄 visualizar_paciente.py  
  ✓ receitas  
    > __pycache__  
    📄 receitas.py  
  📄 main.py
```

Composição e Diagrama

Sintetizando o conceito de Composição, uma classe Consulta não herda de Médico nem de Paciente. A Consulta 'possui' um objeto Paciente e um objeto Médico dentro dela. Eles compõem a consulta.

```
7
8  class Consultas:
9      def __init__(self, paciente, medico, data, horario, data_hora_completa):
10         self.paciente=paciente # <--- composição
11         self.medico=medico # <--- composição
12         self.data=data
13         self.horario=horario
14         self.data_hora_completa = data_hora_completa
15
```

consultas.py

Programação Orientada a Objetos (POO)

Abstração: Classe Especialidade (ABC)

Herança: subclasses herdam o método de Especialidade(ABC), com o intuito de criar novas classes que herdam os mesmos atributos e/ou métodos.

- **Classe Abstrata:** Define um modelo ("contrato"), impedindo a criação de especialidades aleatórias.
- **Método Abstrato (obter_nome()):** Obriga todas as especialidades filhas (Cardiologia, Pediatria, etc.) a fornecerem sua identificação no (obter_nome()).
- **Garantia:** Assegura que o sistema sempre consiga exibir o nome de qualquer especialidade de forma padronizada.

```
medicos > registrar.py > ...
1  from abc import ABC, abstractmethod
2  from rich import print
3  from rich.table import Table
4
5
6  class Especialidade(ABC):
7      @abstractmethod
8      def obter_nome(self) -> str:
9          pass
10
11  class ClinicoGeral(Especialidade):
12      def obter_nome(self):
13          return "Clinico Geral"
14
15  class Cardiologia(Especialidade):
16      def obter_nome(self):
17          return "Cardiologia"
18
19  class Endocrinologia(Especialidade):
20      def obter_nome(self):
21          return "Endocrinologia"
22
23  class Ginecologia(Especialidade):
24      def obter_nome(self):
25          return "Ginecologia"
26
27  class Dermatologia(Especialidade):
28      def obter_nome(self):
29          return "Dermatologia"
30
```

A classe Especialidade define a interface e as classes concretas definem o conteúdo.

Programação Orientada a Objetos (POO)

registrar.py

```
while True:
    rg = input("RG do médico: ")
    if rg.isdigit() and 7 <= len(rg) <= 9:
        break
    print("[red]Erro: O RG deve conter apenas números e ter entre 7 e 9 dígitos.[/red]")

while True:
    cpf = input("CPF do médico: ")
    if cpf.isdigit() and len(cpf) == 11:
        break
    print("[red]Erro: O CPF deve conter exatamente 11 números.[/red]")

while True:
    crm = input("CRM do médico (6 dígitos): ")
    if crm.isdigit() and len(crm) == 6:
        break
    print("[red]Erro: O CRM deve conter exatamente 6 números.[/red]")

try:
    medico = Medico(nome, rg, cpf, crm, opcao_especialidade)
    CadastroMedico.medicos.append(medico)
    print("[green]\nMédico cadastrado com sucesso![/green]")
except ValueError as e:
    print(f"[red]Erro no cadastro do médico! [/red] {e}")
    return
```

Encapsulamento -Regra de Negócio

Agrupamento de dados (Paciente, Medico) e isolamento da lógica de validação.

O objeto só é criado se os dados forem 100% válidos.

E isso é algo que o main.py não acessa diretamente, pois está encapsulado na lógica da instância cadastrarMedico() da classe CadastroMedico.

Programação Orientada a Objetos (POO)

Polimorfismo

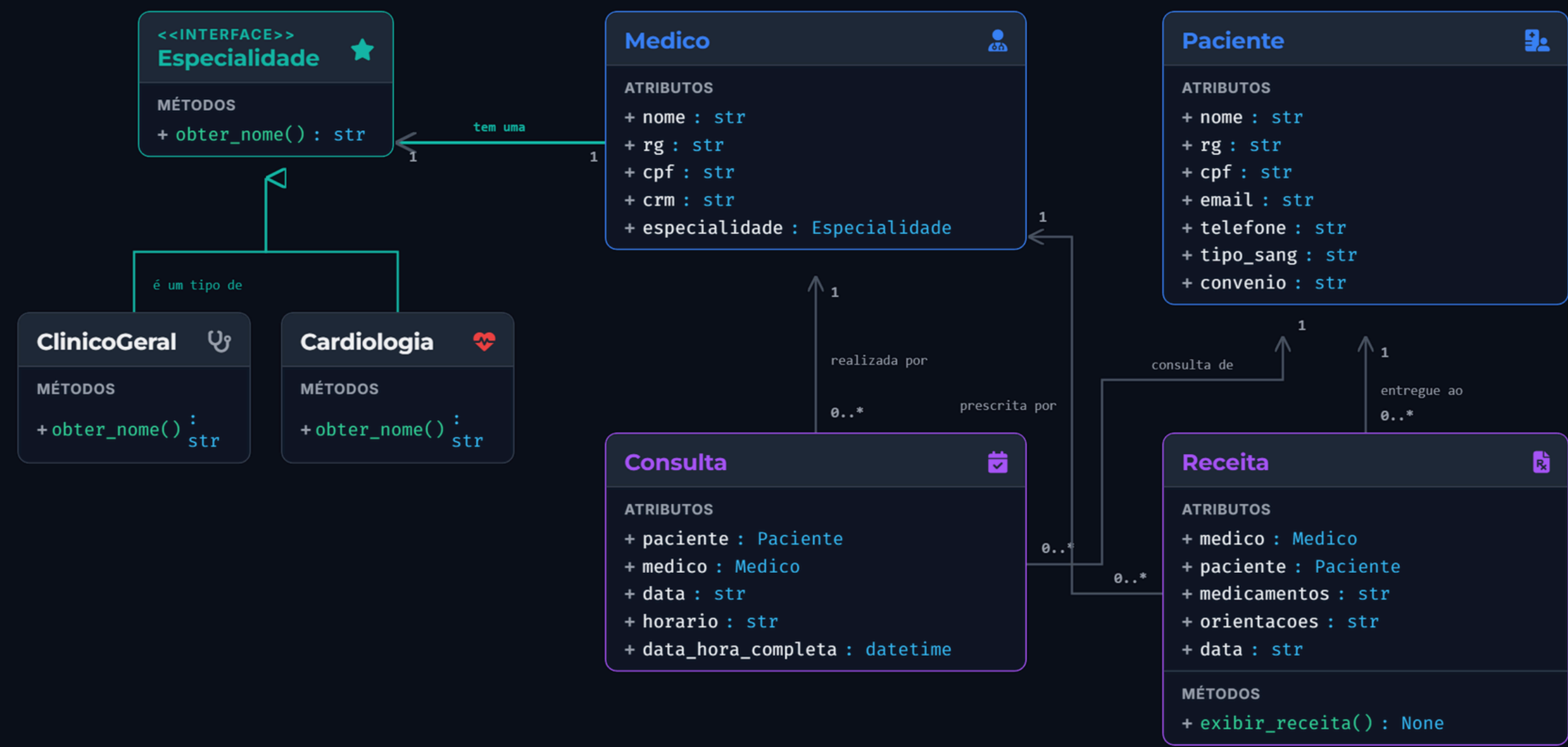
A palavra Polimorfismo significa "muitas formas" (Poli = muitas, Morfos = formas).

Na programação, isso significa que um mesmo comando pode gerar respostas diferentes, dependendo de quem o recebe.

```
class VisualizarMedico:
    @staticmethod
    def exibir_medicos():
        print("\n-_-_-_-_- LISTA DE MÉDICOS -_-_-_-_-")
        for medico in CadastroMedico.medicos:
            print(f"Nome: {medico.nome}")
            print(f"CRM: {medico.crm}")
            print(f"Especialidade: {medico.especialidade.obter_nome()}")
            print("-" * 30)
```

visualizar.py

Diagrama de Classes UML



A arquitetura reflete o mapeamento correto do domínio de negócios e suas restrições:

Relação	Multiplicidade
Paciente — Consulta	1 para 0..* (Paciente possui de 0 a N consultas)
Médico — Consulta	1 para 0..* (Médico realiza de 0 a N consultas)
Médico — Receita	1 para 0..* (Prescrita por 1 médico)
Médico — Especialidade	1 para 1 (Injeção de dependência)

Princípios SOLID

S	Single responsibility principle
O	Open/closed principle
L	Liskov substitution principle
I	Interface segregation principle
D	Dependency inversion principle

Desafios Utilizando O Princípio S.O.L.I.D

Cumprindo os desafios proposto, também aplicamos outros principios que são de extrema importância para o fluxo do programa

S — Single Responsibility Principle (Princípio da Responsabilidade Única)

O — Open/Closed Principle (Princípio Aberto/Fechado)

L — Liskov Substitution Principle (Princípio da Substituição de Liskov)

I — Interface Segregation Principle (Princípio da Segregação de Interface)

D — Dependency Inversion Principle (Princípio da Inversão de Dependência)

Princípios SOLID

Utilizados (Parte 1)

10

SRP (Princípio da Responsabilidade Única):

- Separação estrita entre Entidades (dados e propriedades) e Controles de inputs/buscas.
 - Exemplo: Paciente (dados), CadastroPaciente (input), BuscaPaciente (busca).
- Paciente apenas encapsula os dados e validações da entidade.
- CadastroPaciente trata da interação de inputs com o terminal.
- BuscaPaciente gerencia exclusivamente o fluxo de queries e buscas.

LSP (Princípio da Substituição de Liskov):

- A classe Medico não possui importações ou acoplamento direto com as especialidades (como Cardiologia).
- Ela recebe uma dependência da classe abstrata Especialidade injetada via construtor.

Princípios SOLID

Utilizados (SRP)

11

```
class Paciente:
    def __init__(self, nome, rg, cpf, email, telefone, contato_emergencia, tipo_sang: str = "Não informado"):
        self.nome = nome
        self.rg = rg
        self.cpf = cpf
        self.email = email
        self.telefone = telefone
        self.contato_emergencia = contato_emergencia
        self.tipo_sang = tipo_sang
        self.peso = peso
        self.altura = altura
        self.alergias = alergias
        self.doencas_pre_ex = doencas_pre_ex
        self.med_em_uso = med_em_uso
        self.hist_cirurgias = hist_cirurgias
        self.convenio = convenio
```

paciente.py

A classe Paciente APENAS armazena dados

Princípios SOLID

Utilizados (SRP)

12

paciente.py

```
class CadastroPaciente:
    pacientes = []

    @staticmethod
    def cadastrar_paciente():
        nome = input("Nome do paciente (ou digite '0' para cancelar): ")
        if nome == "0":
            print("[yellow]Cadastro cancelado. Voltando ao menu...[/yellow]")
            return

        while True:
            rg = input("RG do paciente(ou digite '0' para cancelar): ")

            if rg == '0':
                print("[yellow]Cadastro cancelado. Voltando ao menu...[/yellow]")
                return

            if rg.isdigit() and 7 <= len(rg) <= 9:
                break

            print("[red]Erro: O RG deve conter apenas números e ter entre 7 e 9 dígitos.[/red]")

            while True:
                cpf = input("CPF do paciente (ou digite '0' para cancelar): ")
                if cpf.isdigit() and len(cpf) == 11:
                    break
                print("[red]Erro: O CPF deve conter exatamente 11 números.[/red]")

            email = input("E-mail: ")
            telefone = input("Telefone / Celular: ")
            contato_emergencia = input("Contato de emergência: ")

            tipo_sang = input("Tipo Sanguíneo [A+, A-, B+, B-, AB+, AB-, O+, O-] (ou Enter para pular): ") or "Não informado"
```

Enquanto a classe CadastroPaciente, possui um método para realizar o cadastro

Princípios SOLID

Utilizados (Parte 2)

OCP (Princípio Aberto/Fechado):

- A classe Especialidade está fechada para modificações, mas aberta para extensão.
- Para adicionar novas especialidades, basta herdar sem modificar o código do cadastro de médicos.
- Para adicionar uma nova especialidade (ex: Pediatria), basta herdar a classe abstrata Especialidade.
- Nenhum código existente no menu principal ou no cadastro de médicos precisa ser alterado.

DIP (Princípio da Inversão de Dependência):

- Classe Medico depende da interface genérica Especialidade e não de especialidades concretas diretamente.
- Qualquer classe concreta (ex: ClinicoGeral ou Cardiologia) substitui Especialidade.
- O sistema chama `medico.especialidade.obter_nome()` com polimorfismo puro e transparente.

Princípios SOLID Utilizados (OCP)

14

Open-Closed Principle

Aberto para extensão, mas fechado
para modificação.

Caso a clínica passe a ter atendimento
neurológico:

registrar.py

```
class Ortopedia(Especialidade):  
    def obter_nome(self):  
        return "Ortopedia"  
  
class Pediatria(Especialidade):  
    def obter_nome(self):  
        return "Pediatria"  
  
class Neurologia(Especialidade):  
    def obter_nome(self):  
        return "Neurologia"
```

Princípios SOLID

Utilizados (LSP)

Princípio da Substituição de Liskov

```
class Medico:
    def __init__(self, nome:str , rg:str, cpf:str, crm:str, especialidade: Especialidade):
        self.nome = nome
        self.rg=rg
        self.cpf=cpf
        self.crm = crm
        self.especialidade = especialidade
```

registrar.py

Nesse sistema (o `__init__`) está engessado, esperando receber estritamente o molde, que é a especialidade da classe Especialidade.

```
# TESTE -----
if __name__ == "__main__":
    medico_teste= CadastroMedico.cadastrarMedico()
    CadastroMedico.medicos.append(Medico("Dr. House", "1234567", "12345678901", "666666", ClinicoGeral()))
```

registrar.py

Aqui, um objeto do tipo ClinicoGeral() é criado e atribuído ao Dr. House

Princípios SOLID

Utilizados (ISP)

16

Princípio de Segregação de Interfaces

```
registrar.py
class Especialidade(ABC):
    @abstractmethod
    def obter_nome(self) -> str:
        pass

class ClinicoGeral(Especialidade):
    def obter_nome(self):
        return "Clinico Geral"
```

(-> str) Type hinting (String)

Princípios SOLID Utilizados (DIP)

17

Princípio da Inversão de Dependência

```
class Medico:
    def __init__(self, nome:str , rg:str, cpf:str,.crm:str, especialidade: Especialidade):
        self.nome = nome
        self.rg=rg
        self.cpf=cpf
        self.crm = crm
        self.especialidade = especialidade
```

registrar.py

O construtor da nossa classe Medico recebe o parâmetro especialidade: Especialidade. O médico depende de uma abstração, e não de uma classe concreta como ClinicoGeral. Essa inversão de dependência é que permite que o sistema seja altamente flexível.

Conclusão e Próximos Passos

Alta Flexibilidade:

O sistema está pronto para substituir o terminal por um banco de dados persistente (PostgreSQL/SQLite) alterando poucas linhas nos arquivos de controle.

Robustez:

O encapsulamento eliminou estados inválidos na criação de dados.

Legibilidade e Manutenibilidade:

Desacoplamento de responsabilidades permitindo que o sistema cresça com facilidade e novas classes sejam testadas isoladamente.

HORA DE RODAR O PROGRAMA.